WEB ENGINEERING · CSE601330

Flask Project Rubric

Grading Criteria for Flask-Based Web Applications

Covers Weeks 5–8: Routing, Templates, SQLAlchemy CRUD, Forms & Sessions

Target Audience: BS (Data Science) / BS (Artificial Intelligence)
Total Points: 100

Course: Web Engineering (CSE601330) | **Technology:** Flask (Weeks 5–8) | **Total Points:** 100

Flask Project Grading Rubric

Project Overview

This rubric evaluates a Flask-based web application demonstrating the core back-end skills covered in Weeks 5–8: routing and view functions, Jinja2 templating with inheritance, a SQLAlchemy-backed database with full CRUD operations, server-side form validation, and session/cookie-based state management. It applies to any project built to this scope — a student portal, library system, booking tool, or similar CRUD-driven application — regardless of the specific domain chosen.

Minimum Database Scope (Policy): The project's database must define a minimum of **40 tables (models)**, connected through meaningful relationships rather than left standalone. A schema below this minimum is capped at the Needs Improvement band for Database Integration & CRUD, regardless of how well the implemented tables otherwise meet the criterion.

Grading Criteria

Criterion	Excellent	Satisfactory	Needs Improvement
Project Setup & Structure (10 pts)	(9–10) Virtual environment used correctly; `requirements.txt` present and accurate; files organized following the conventional structure (`app.py`, `templates/`, `static/`), or a blueprint/application factory structure for larger projects	(5–8) Project runs but structure is inconsistent, `requirements.txt` is missing some dependencies, or files are only partially organized	(0–4) No virtual environment evidence, no `requirements.txt`, or files are disorganized to the point of being hard to navigate
Routing & View Functions (15 pts)	(13–15) Multiple well-designed routes with correct HTTP methods, appropriate use of variable path segments and converters (e.g., `<int:id>`), and clean, single-responsibility view functions	(7–12) Routes work but methods are occasionally misused (e.g., GET used where POST is appropriate), or view functions mix unrelated concerns	(0–6) Routes are missing, broken, or too few to demonstrate the required functionality

Criterion	Excellent	Satisfactory	Needs Improvement
Templates & Jinja2 Usage (15 pts)	(13–15) Template inheritance (`{% extends %}` / `{% block %}`) used consistently; loops and conditionals applied correctly; no duplicated layout code across pages	(7–12) Templates render correctly but inheritance is partial, or some layout code (header/nav/footer) is duplicated instead of shared	(0–6) No template inheritance used, or templates contain significant logic errors / broken Jinja2 syntax
Database Integration & CRUD (20 pts)	(17–20) At least 40 SQLAlchemy models (tables) correctly defined with appropriate columns, constraints, and meaningful relationships between them; all four CRUD operations (Create, Read, Update, Delete) implemented and working correctly across the schema	(10–16) Fewer than 40 tables, or the 40-table minimum is met but relationships are misconfigured, one CRUD operation is missing, or queries are inefficient/incorrect in places	(0–9) Database schema falls well short of the 40-table minimum; models are missing or broken; most CRUD operations are absent or non-functional
Forms, Validation, Cookies & Sessions (15 pts)	(13–15) Server-side validation implemented beyond HTML5 attributes; cookies and/or sessions used appropriately; flash messages and the Post-Redirect-Get pattern applied correctly where relevant	(7–12) Forms work but validation is client-side only, or sessions/cookies are used but with minor security or logic issues (e.g., no PRG pattern on a form submission)	(0–6) No server-side validation; sessions/cookies not implemented or implemented incorrectly
Code Quality & Organization (10 pts)	(9–10) Code is readable, consistently formatted, meaningfully named, and organized (blueprints or a clear single-file structure appropriate to project size); no significant duplication	(5–8) Code is functional but inconsistently formatted, has unclear naming, or has noticeable duplication	(0–4) Code is difficult to follow, poorly organized, or shows little attention to structure
Functionality & Testing (10 pts)	(9–10) Application runs without errors; all described features work as intended when tested end to end	(5–8) Application runs with minor bugs or edge cases that fail, but core functionality works	(0–4) Application fails to run, crashes frequently, or core features do not work
Documentation (5 pts)	(5) A clear `README.md` explains the project's purpose, setup, and how to run it, including any environment variables or seed data needed	(3–4) A README exists but is incomplete or unclear in places	(0–2) No README, or documentation is missing entirely

Grade Interpretation

Score Range	Grade Band
90–100	Excellent — demonstrates mastery of Flask routing, templating, ORM-based CRUD, and session handling
75–89	Good — solid working application with minor gaps in one or two areas
60–74	Satisfactory — core functionality present but with notable weaknesses in structure, validation, or completeness
Below 60	Needs Improvement — significant functionality missing or application does not run reliably

Additional Notes

- **Minimum 40 tables:** The database schema must contain at least 40 related tables/models; this is a hard submission requirement, not a stretch goal, and is checked as part of grading the Database Integration & CRUD criterion.
- Deadline, submission format, and any group-work policy should be communicated separately by the instructor as part of the assignment brief.
- Academic integrity: submitted code should represent the student's (or group's) own work; any reused code (e.g., boilerplate from course materials) should be acknowledged.